

力扣一百合集

必备头文件

```
#include <iostream>      // cin, cout
#include <vector>         // vector
#include <string>         // string
#include <algorithm>      // sort, max, min, reverse
#include <queue>          // queue, priority_queue
#include <stack>          // stack
#include <deque>          // deque
#include <set>            // set, multiset
#include <map>            // map, multimap
#include <unordered_set>  // unordered_set
#include <unordered_map> // unordered_map
#include <cmath>          // sqrt, pow, abs
#include <climits>        // INT_MAX, INT_MIN
#include <utility>        // pair, make_pair
#include <cstring>        // memset

#include <bits/stdc++.h>
using namespace std;

// 默认是大顶堆
priority_queue<int> max_heap;

// 小顶堆定义方式
priority_queue<int, vector<int>, greater<int>> min_heap;
```

哈希

两数之和

1. 两数之和

已解答 

简单

 相关标签

 相关企业

 提示

Ax

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出和为目标值 `target` 的那 **两个** 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        int n = nums.size();
        vector<int> ans;
        bool flag = true;
        for(int i = 0; i < n && flag; i++) {
            for (int j = 0; j < n && flag; j++) {
                if (i == j) continue;
                if (nums[i] + nums[j] == target) {
                    ans.push_back(i);
                    ans.push_back(j);
                    flag = false;
                    break;
                }
            }
        }
        return ans;
    }
};
```

49. 字母异位词分组

49. 字母异位词分组

已解答 ✓

中等

🏷 相关标签

🏢 相关企业

Ax

给你一个字符串数组，请你将 **字母异位词** 组合在一起。可以按任意顺序返回结果列表。

示例 1:

输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

输出: [["bat"],["nat","tan"],["ate","eat","tea"]]

解释:

- 在 strs 中没有字符串可以通过重新排列来形成 **"bat"**。
- 字符串 **"nat"** 和 **"tan"** 是字母异位词，因为它们可以重新排列以形成彼此。
- 字符串 **"ate"**，**"eat"** 和 **"tea"** 是字母异位词，因为它们可以重新排列以形成彼此。

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> m;
        for (string &s : strs) {
            string s2 = s;
            sort(s2.begin(), s2.end());
            m[s2].push_back(s); // 排序后相同的分到一起
        }
        vector<vector<string>> ans;
        for (auto it = m.begin(); it != m.end(); it++) {
            vector<string> g = it->second;
            ans.push_back(g);
        }
    }
};
```

```
        return ans;
    }
};
```

128. 最长连续序列

128. 最长连续序列

已解答 

中等

 相关标签

 相关企业

Aa

给定一个未排序的整数数组 `nums`，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

示例 1:

输入: `nums = [100,4,200,1,3,2]`

输出: 4

解释: 最长数字连续序列是 `[1, 2, 3, 4]`。它的长度为 4。

示例 2:

输入: `nums = [0,3,7,2,5,8,4,6,0,1]`

输出: 9

示例 3:

输入: `nums = [1,0,1,2]`

输出: 3

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> num_set;
        for (int num : nums) {
            num_set.insert(num);
        }
    }
};
```

```

    }
    int ans = 0;
    for (int num : num_set) {
        // 如果 num - 1不存在, num为序列的起点
        if (!num_set.count(num - 1)) {
            int curNum = num;
            int curL = 1;
            while(num_set.count(curNum + 1)) {
                curNum ++;
                curL ++;
            }
            ans = max(ans, curL);
        }
    }
    return ans;
}
};

```

双指针

283. 移动零

283. 移动零

已解答 

简单

 相关标签

 相关企业

 提示

Aa

给定一个数组 `nums`，编写一个函数将所有 `0` 移动到数组的末尾，同时保持非零元素的相对顺序。

请注意，必须在不复制数组的情况下原地对数组进行操作。

示例 1:

输入：nums = [0,1,0,3,12]

输出：[1,3,12,0,0]

```

class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        // 双指针
        int j = 0;
        for (int i = 0; i < nums.size(); i++) {

```

```

        if (nums[i] != 0) {
            nums[j] = nums[i];
            j ++;
        }
    }
    while (j < nums.size()) {
        nums[j] = 0;
        j ++;
    }
}
};

```

11. 盛最多水的容器

11. 盛最多水的容器

已解答 

中等

 相关标签

 相关企业

 提示

Ax

给定一个长度为 n 的整数数组 `height`。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, \text{height}[i])$ 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。

说明：你不能倾斜容器。

```

class Solution {
public:
    int maxArea(vector<int>& height) {
        int j = height.size()-1;
        int i = 0;
        int ans = 0;
        while(i <= j){
            int p = min(height[i], height[j]);
            int tp = p * (j - i);
            ans = max(ans, tp);
            if (height[i] < height[j]) {
                i ++;
            }
            else{
                j --;
            }
        }
        return ans;
    }
};

```

```
}  
};
```

15.三数之和

15. 三数之和

已解答 

中等

 相关标签

 相关企业

 提示

⌵

给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k`，同时还满足 `nums[i] + nums[j] + nums[k] == 0`。请你返回所有和为 0 且不重复的三元组。

注意：答案中不可以包含重复的三元组。

```
class Solution {  
public:  
    /*  
     * 思路：  
     * 1. 先对数组排序，方便使用双指针并进行去重。  
     * 2. 枚举第一个数 nums[i]，然后在右侧区间使用双指针 j、k。  
     * 3. 如果三数之和 > 0，说明和太大，k 左移。  
     * 4. 如果三数之和 < 0，说明和太小，j 右移。  
     * 5. 如果三数之和 == 0，记录答案，并跳过重复元素。  
     * 6. 对 i 也需要去重，避免产生重复三元组。  
     * 7. 可以利用排序后的大小关系进行剪枝，减少无效搜索。  
     *  
     * 时间复杂度：O(n^2)  
     */  
    vector<vector<int>> threeSum(vector<int>& nums) {  
        sort(nums.begin(), nums.end());  
        vector<vector<int>> ans;  
        int n = nums.size();  
        for (int i = 0; i < n - 2; i++)  
        {  
            int x = nums[i];  
            if (i && x == nums[i - 1]) continue; // 跳过重复数字  
            if (x + nums[i + 1] + nums[i + 2] > 0) break;  
            if (x + nums[n - 1] + nums[n - 2] < 0) continue;  
            int j = i + 1, k = n - 1;  
            while(j < k) {  
                int s = x + nums[j] + nums[k];  
                if (s > 0) {  
                    k --;  
                }  
                else if (s < 0) {
```

```

        j ++;
    }
    else{
        ans.push_back({x, nums[j], nums[k]});
        for (j ++; j < k && nums[j] == nums[j - 1]; j ++);
        for (k-- ; k > j&& nums[k] == nums[k + 1]; k --);
    }
}
}
return ans;
}
};

```

42. 接雨水

42. 接雨水

已解答 

困难

 相关标签

 相关企业

Ax

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

```

class Solution {
public:
    int trap(vector<int>& height) {
        if (height.size() <= 2) return 0;
        int n = height.size();
        int l = 0, r = n - 1;
        int leftMax = 0, rightMax = 0;
        int ans = 0;
        while(l < r){
            leftMax = max(leftMax, height[l]);
            rightMax = max(rightMax, height[r]);
            if (leftMax < rightMax) {
                ans += leftMax - height[l];
                l ++;
            }
            else{
                ans += rightMax - height[r];
                r --;
            }
        }
        return ans;
    }
};

```


滑动窗口

无重复字符的最长子串

3. 无重复字符的最长子串

已解答 

中等

 相关标签

 相关企业

 提示

Ax

给定一个字符串 `s`，请你找出其中不含有重复字符的 **最长子串** 的长度。

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        // 用哈希表记录每个字符上一次出现的位置
        // 当发现当前字符在窗口中出现过，就把left移动到它上一次出现位置的最后一位
        unordered_map<char, int> last;
        int l = 0;
        int ans = 0;
        for (int r = 0; r < s.size(); r++)
        {
            char c = s[r];
            while (last.count(c) && last[c] >= l)
            {
                l = last[c] + 1;
            }
            last[c] = r;
            ans = max(ans, r - l + 1);
        }
        return ans;
    }
};
```

438. 找到字符串中所有字母异位词

已解答 ✓

中等

🏷 相关标签

🏢 相关企业

Aa

给定两个字符串 `s` 和 `p`，找到 `s` 中所有 `p` 的 **异位词** 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

示例 1:

输入：s = "cbaebabacd", p = "abc"

输出：[0,6]

解释：

起始索引等于 0 的子串是 "cba"，它是 "abc" 的异位词。

起始索引等于 6 的子串是 "bac"，它是 "abc" 的异位词。

```
class Solution {
public:
    /*
     * 思路：滑动窗口 + 字符计数
     * 1. 异位词只要求字符种类和每个字符出现次数相同，因此用长度为 26 的数组统计字符频率。
     * 2. 先统计 p 的字符频率 cntP，以及 s 中前 m 个字符组成窗口的频率 cntS。
     * 3. 如果 cntS == cntP，说明当前窗口是 p 的异位词，记录窗口起点。
     * 4. 之后窗口每次向右移动一格：
     *     - 加入右侧新字符 s[i]
     *     - 删除左侧移出的字符 s[i - m]
     * 5. 每次移动后比较 cntS 和 cntP，相同则记录当前窗口起点。
     */
    vector<int> findAnagrams(string s, string p) {
        int n = s.size(), m = p.size();
        vector<int> ans;
        if (n < m) return ans;
        vector<int> cntS(26, 0);
        vector<int> cntP(26, 0);
        for (int i = 0; i < m; i++)
        {
            cntS[s[i] - 'a']++;
            cntP[p[i] - 'a']++;
        }
        if (cntS == cntP) ans.push_back(0);
        for (int i = m; i < n; i++)
```

```
    {  
        cntS[s[i] - 'a'] ++;  
        cntS[s[i - m] - 'a'] --;  
        if (cntS == cntP) ans.push_back(i - m + 1);  
    }  
    return ans;  
}  
};
```

子串
